File 05 - AgileFlow AI Implementation Working Plan

Project foundation, execution order, team coordination, AI-agent rules, and integration workflow

Purpose of this document

This file is a knowledge-passing document for starting the implementation of AgileFlow AI. It explains how the group leader should initialize the project, split the work, control branches, guide AI-agent-assisted development, and avoid missing functions or integration conflicts.

1. Implementation Strategy

AgileFlow AI should be implemented as a web-based research prototype. The first goal is not to build a perfect commercial product. The first goal is to prove the full SDLC workflow from a raw client idea to a validated prototype foundation with human approval gates and traceability evidence.

- Start with a working end-to-end prototype using one controlled sample domain such as a book-management system or appointment-booking system.
- Use mock or template-generated outputs first, then replace them gradually with AI-generated outputs and real tool integrations.
- Build the common project foundation before allowing members to implement separate agents.
- Keep the Orchestrator and Shared Knowledge Base as the central connection points for all components.
- Every feature must support traceability IDs and approval status from the beginning.

2. Recommended Technology Stack

Layer	Recommended Technology	Reason
Frontend	React + Vite + Tailwind CSS	Fast dashboard development, reusable components, strong fit for a SaaS-style research prototype.
Backend	Python FastAPI	Simple API creation, good support for AI workflows, JSON schemas, automation, and testing.
Database	MongoDB	Flexible JSON-based storage for requirements, designs, tests, logs, reports, and traceability links.
Storage	Local project storage first	Simple for MVP; can store uploads, generated artifacts, logs, and reports per project.
AI Layer	Prompt templates + structured JSON	Feasible for final-year prototype;

	outputs	avoids unnecessary model training at the start.
Execution Layer	Mock execution first, Docker later	Supports a staged approach from simple prototype to real build/run validation.
Integrations	Mock Jira/Git first, real integration later	Reduces early complexity and avoids blockers during foundation development.

3. Initial Team Split for Starting Phase

At the beginning, the group can be split into two sub-groups. This is a practical approach because the UI design and project foundation can progress in parallel without blocking each other.

Sub Group	Members	Responsibility	Main Output
Group A - UI Design Team	Two members	Design Figma screens, user flow, dark purple UI theme, reusable components, approval screens, and report pages.	Complete UI prototype and component handoff notes.
Group B - Project Foundation Team	Group leader + one member	Initialize repository, frontend, backend, database, documentation, folder structure, API contracts, schemas, and branch rules.	Working base project with setup instructions and initial APIs.

Important rule

During the first few days, the UI team should mainly work in Figma or design documents. They should not create a separate frontend structure until the foundation team defines the approved frontend folders and component structure.

4. Repository and Folder Structure

The group leader should initialize one monorepo so frontend, backend, documentation, sample data, and generated artifacts stay in one controlled location.

Recommended monorepo structure

```
agileflow-ai/
|-- frontend/
|-- backend/
|-- docs/
|   |-- architecture/
|   |-- api-contracts/
|   |-- schemas/
|   |-- ai-agent-rules/
|   |-- meeting-notes/
|   |-- research/
```

```
|-- storage/
|   |-- uploads/
|   |-- generated-artifacts/
|   |-- logs/
|   |-- reports/
|-- sample-data/
|-- README.md
|-- CONTRIBUTING.md
|-- AI_AGENT_RULES.md
|-- TASK_ASSIGNMENT.md
```

Backend module structure

FastAPI structure

```
backend/app/
|-- main.py
|-- core/
|-- database/
|-- models/
|-- schemas/
|-- modules/
|   |-- projects/
|   |-- orchestrator/
|   |-- approvals/
|   |-- requirements/
|   |-- design/
|   |-- implementation/
|   |-- testing/
|   |-- traceability/
|   |-- reports/
```

Frontend module structure

React structure

```
frontend/src/
|-- components/
|   |-- common/
|   |-- workflow/
|   |-- approvals/
|   |-- agent-panels/
|-- layouts/
|-- pages/
|   |-- dashboard/
|   |-- projects/
```

```
|   |-- requirements/
|   |-- design/
|   |-- testing/
|   |-- implementation/
|   |-- orchestrator/
|   |-- reports/
|-- services/
|-- routes/
|-- utils/
```

5. Branching and Merge Plan

Branching rules are needed because all members may use AI coding agents. The develop branch must always remain runnable.

- main - stable final version only.
- develop - integration branch; must always build and run.
- feature/project-foundation - initial repository, backend, frontend, docs, schemas, and base workflow.
- feature/ui-design-system - UI layout, shared components, theme, and Figma-to-frontend implementation.
- feature/agent-requirements - Requirement Gathering and Analysis Agent.
- feature/agent-design-planning - Project Management and Design Agent.
- feature/agent-implementation - Implementation and Git Automation Agent.
- feature/agent-testing-validation - Testing, Validation and Security Agent.
- feature/orchestrator - workflow control, approval gates, routing, and retry logic.
- feature/reports-traceability - final reports and traceability matrix.

Branch safety rule

Incomplete feature branches are acceptable. A broken develop branch is not acceptable.

6. Implementation Phases

Phase	Focus	Main Tasks	Exit Criteria
Phase 1	Foundation setup	Create repository, folder structure, docs, frontend, backend, database connection, project model, status model, approval model.	Frontend and backend run locally; project can be created.
Phase 2	Workflow and Orchestrator	Implement workflow stages, approval gates, next-step logic, event history, and Shared Knowledge Base structure.	Project can move through H1-H5 approval states.
Phase 3	Agent mock outputs	Each member builds assigned agent screen and mock API output using approved schemas.	End-to-end flow works with sample data.
Phase 4	Real logic and storage	Save outputs to database, connect agent outputs, create traceability links, and generate reports.	Outputs persist and are visible across components.
Phase 5	Integration and evaluation	Connect feedback loops, test workflow, fix bugs, prepare screenshots, evidence, and demo.	Full demo from client idea to final validation report works.

7. First Sprint Task Order

The first sprint should focus on creating a stable base. Agent features should not be fully developed before the common foundation is ready.

1. Create GitHub repository and protected main/develop branches.
2. Create documentation files: README, CONTRIBUTING, AI_AGENT_RULES, API_CONTRACT, DATA_SCHEMAS, TASK_ASSIGNMENT, UI_PAGES.
3. Setup React + Vite + Tailwind frontend.
4. Setup FastAPI backend and environment variables.
5. Setup MongoDB connection and database helper.
6. Create Project model, Approval model, WorkflowEvent model, and TraceabilityLink model.
7. Create basic project APIs and dashboard UI.
8. Create reusable frontend layout: sidebar, top bar, project selector, status badges, approval cards.
9. Create Orchestrator next-step logic for H1-H5 stages.
10. Prepare one sample project and demo data for all agents.

8. Member-Wise Implementation Ownership

Member / Component	Branch	Primary Tasks	Output	Do Not Modify
Requirement Gathering and Analysis Agent	feature/agent-requirements	Requirement input, extraction output, SRS summary, user stories, acceptance criteria, ambiguity report, H1 approval.	Approved requirement package with REQ IDs.	Design, implementation, and testing modules.
Project Management and Design Agent	feature/agent-design-planning	Backlog, sprint plan, API plan, DB schema, UI screen list, requirement-to-design mapping, H2 approval.	Approved design and planning package.	Requirement extraction, code generation, and final validation logic.
Implementation and Git Automation Agent	feature/agent-implementation	Generated project structure, code modules, Git metadata, build/run logs, limited self-fix flow, H4 approval.	Prototype foundation with code metadata and logs.	Requirement processing, design generation, and final validation schema without approval.
Testing, Validation and Security Agent	feature/agent-testing-validation	Pre-code tests, H3 approval, test results, bug reports, security findings, H5 approval.	Test package, validation report, bug report, security report.	Agent 3 generation logic and Agent 2 planning logic.

9. AI Agent Development Rules

Because the team will use AI agents to write code, each prompt must restrict what the AI is allowed to change. This prevents accidental edits, duplicate files, and broken integrations.

- Always mention the current branch and assigned component.
- Always list allowed files and folders before asking the AI to code.

- Always list do-not-change files and protected shared files.
- Ask the AI to follow README, API_CONTRACT, DATA_SCHEMAS, and AI_AGENT_RULES.
- Do not allow the AI to rename API endpoints, shared models, or status values without approval.
- Do not allow the AI to create duplicate schemas or duplicate service functions.
- Do not allow the AI to delete or rewrite another member's component.
- After every AI-generated change, review changed files manually before committing.

Reusable AI coding prompt template

You are working on AgileFlow AI.

Component: [component name]
Current branch: [branch name]
Allowed files/folders: [list]
Do not modify: [list]
Follow these documents: README.md, API_CONTRACT.md, DATA_SCHEMAS.md, AI_AGENT_RULES.md

Task: [clear task]

Requirements:
- Use existing shared models.
- Do not create duplicate schemas.
- Do not rename existing APIs.
- Do not change unrelated files.
- Keep code simple and readable.
- Return a summary of changed files.

10. Required Shared Status Values and IDs

All components must use the same statuses, approval gates, and IDs. This is essential for traceability and workflow control.

Category	Allowed Values / Format	Usage
Workflow status	PENDING, IN_PROGRESS, WAITING_FOR_APPROVAL, APPROVED, REJECTED, FAILED, COMPLETED	Used by projects, agents, approvals, and workflow stages.
Approval gates	H1, H2, H3, H4, H5	Requirement approval, design approval, pre-code test approval, implementation approval, final validation approval.
Requirement IDs	REQ-001, REQ-002	Used by Agent 1 and traceability matrix.
Design IDs	DES-001, API-001, DB-001, UI-001	Used by Agent 2 and design traceability.
Test IDs	TEST-001, SEC-001	Used by Agent 4 and requirement-to-test mapping.
Code IDs	CODE-001	Used by Agent 3 and requirement-to-code mapping.
Bug IDs	BUG-001	Used by Agent 4 feedback and Agent 2 issue tracking.

11. Pull Request and Review Rules

- Every member must create a feature branch from develop.
- Every feature branch must be updated from develop before creating a pull request.
- Pull request target must be develop, not main.
- PR title format: [Component Name] Short task description.
- PR description must include changed files, APIs added/updated, database collections used, testing steps, screenshots for UI changes, and known issues.
- At least one other member or the group leader must review before merge.
- If a PR changes shared schemas or APIs, the API_CONTRACT and DATA_SCHEMAS documents must be updated in the same PR.

12. Conflict Prevention Checklist

Protected files

- backend/app/main.py
- backend/app/core/config.py
- backend/app/database/*
- backend/app/models/common.py
- frontend/src/App.jsx or App.tsx
- frontend/src/routes/*
- frontend/src/layouts/MainLayout.*
- docs/API_CONTRACT.md
- docs/DATA_SCHEMAS.md

Only the group leader or assigned foundation owner should modify protected files. Members should request changes instead of directly editing them.

Before merging, check these

- Backend starts without errors.
- Frontend starts without errors.
- No API route name was changed without documentation.
- No duplicate models or duplicate services were created.
- No member files were deleted or renamed accidentally.
- Status values and approval gates follow the agreed constants.
- Traceability IDs are preserved.
- Screenshots or test evidence are attached for completed UI/API work.

13. MVP Demonstration Flow

The first MVP should show a complete end-to-end workflow using one sample software idea. The prototype can use generated mock outputs at first, but the workflow must be real and connected.

11. Create project.
12. Enter client idea.
13. Requirement Gathering and Analysis Agent generates requirements.
14. Human approves H1.
15. Project Management and Design Agent generates backlog and design artifacts.
16. Human approves H2.
17. Pre-Code Test Planning Agent generates test specifications.
18. Human approves H3.
19. Implementation and Git Automation Agent generates prototype files and logs.
20. Human approves H4.
21. Testing, Validation and Security Agent generates validation, bug, and security reports.
22. Human approves H5.
23. Final traceability report is displayed.

14. Weekly Execution Plan

Week	Focus	Expected Result
Week 1	Foundation setup, documentation, UI design, folder structure, database, project model, workflow and approval base.	Repository and base application are ready.
Week 2	Agent screens, reusable UI components, mock APIs, and sample outputs.	All main screens and mock workflow are visible.
Week 3	Backend storage, Shared Knowledge Base, Orchestrator connection, approval flow, and traceability base.	Agent outputs are saved and connected.
Week 4	Component integration, feedback loops, reports, bug/security simulation, and traceability matrix.	Full workflow works from start to final report.

Week 5	Testing, fixing, screenshots, research evidence, demo preparation, and evaluation data.	Stable prototype ready for presentation or supervisor review.

15. Group Leader Daily Checklist

- Check whether develop branch is still runnable.
- Check open PRs and whether they follow documentation rules.
- Confirm that no shared schema or API changed without approval.
- Update task board and assign next tasks clearly.
- Resolve conflicts early instead of waiting until the end.
- Collect screenshots, logs, and evidence for research documentation.
- Make sure every member writes what they changed and what remains to complete.

Final implementation principle

Build a working workflow first, not perfect AI. First complete UI + APIs + database + workflow + approval gates + mock outputs. Then improve AI generation, Git automation, Docker execution, testing, security scanning, and traceability depth.